



Voltik

Monitorização preditiva de glicemia, com a assistente Iris

Relatório de Viabilidade Tecnológica

Unidade curricular

Projeto de Tecnologias e Sistemas de Informação & Integração de Sistemas e Aplicações

Instituição

Instituto Politécnico da Maia (IPMAIA) — CTeSP TPSI

Elementos do grupo

**Rui Passos — A048863
Afonso Carvalho — A049948**

Data de entrega

1 de outubro de 2026



Índice

1. Objetivo	3
2. Metodologia adotada	3
3. Tecnologias previstas	3
4. Arquitetura da solução	4
5. Estudo de viabilidade tecnológica	5
6. Exemplo prático: servidor na Oracle Cloud	6
7. Modelo de dados	6
8. Custos	8
9. Riscos e mitigação	8
10. Conclusão	9



1. Objetivo

Este relatório demonstra que o Voltik é viável com as tecnologias escolhidas, os recursos disponíveis e uma equipa de dois elementos. Apresenta a metodologia de trabalho, a arquitetura da solução, as tecnologias e a justificação de cada uma, o modelo de dados, a infraestrutura criada, os custos e os principais riscos. As funcionalidades estão descritas no Caderno de Encargos, entregue em separado.

2. Metodologia adotada

O projeto segue **Scrum adaptado a uma equipa de dois elementos**. Com apenas dois membros, os papéis de Product Owner, Scrum Master e equipa de desenvolvimento acumulam-se, pelo que as cerimónias foram simplificadas em vez de replicadas por inteiro.

Elemento	Aplicação no projeto
Sprint	Uma semana, para ter retorno rápido e acomodar horários de estudo e trabalho
Cerimónias	Planeamento, revisão e retrospectiva numa única sessão semanal; daily substituída por uma mensagem escrita por dia (feito, a fazer, bloqueios)
Backlog	Épicos, histórias e subtarefas no Jira, priorizados por MoSCoW (Must, Should, Could, Won't)
Estimativa	Horas ideais nas primeiras sprints; story points quando existir velocidade histórica
Quadro	Kanban com limite de 2 tarefas em curso por pessoa
Código	GitHub: cada tarefa num ramo próprio, com pull request revisto pelo outro elemento antes de entrar no ramo principal
Divisão de trabalho	Desenvolvimento colaborativo integral e paritário (50/50) entre Afonso Carvalho e Rui Passos em todas as componentes do ecossistema: Aplicação Móvel, Portal Web, Arquitetura da API em Go, Modelação de Bases de Dados, Integração do Modelo de IA Iris, Infraestrutura Cloud em Linux (Oracle Cloud & Docker) e Automação de Integração Contínua (Jenkins).

3. Tecnologias previstas

Componente	Tecnologia
Aplicação Android	Kotlin com Jetpack Compose, desenvolvida no Android Studio
Aplicação iOS	Swift com SwiftUI, desenvolvida no Xcode
Portal web	Vue 3
Backend	Go, API REST documentada em OpenAPI
Modelo de previsão	Python com FastAPI; versão leve executada no telemóvel
Base de dados	PostgreSQL na cloud (29 tabelas); SQLite no dispositivo
Base de dados alimentar	FatSecret Platform API

Componente	Tecnologia
Autenticação	OAuth 2.0 / OpenID Connect: Microsoft, Google, Apple e Facebook
Assistente Iris	API Google Gemini
Sensor de glicemia	Bluetooth Low Energy
Wearables	Health Connect (Android) e HealthKit (iOS)
Notificações	Firebase Cloud Messaging (Android) e APNs (iOS)
Teleconsultas	WebRTC
Alojamento	Oracle Cloud Infrastructure, Ubuntu Server LTS, Docker
Integração contínua	Jenkins
Gestão do projeto	Jira e GitHub

4. Arquitetura da solução

A arquitetura é **offline-first**: a leitura do sensor, a previsão e os alertas acontecem no próprio telemóvel, para funcionarem sem Internet. A cloud guarda o histórico, gera relatórios e serve o portal web, o modo cuidador e as teleconsultas, mas não está no caminho crítico de um alerta.

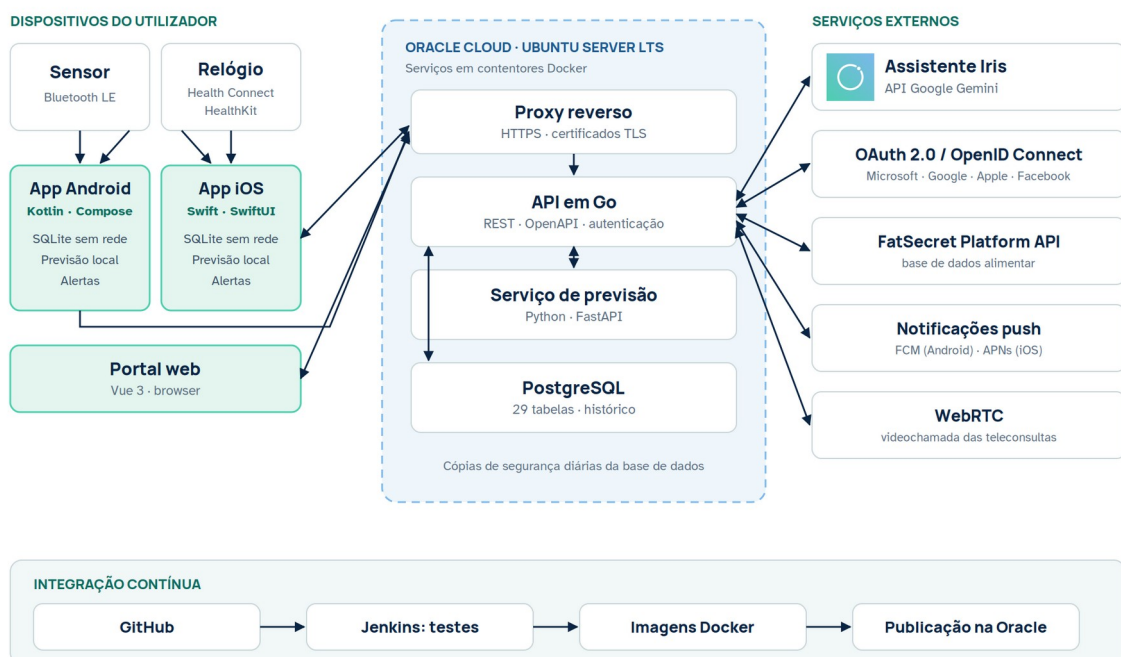
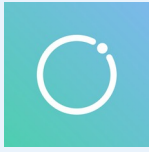


Figura 1 — Arquitetura da solução

Enquanto não há rede, as leituras ficam numa fila em SQLite. Cada leitura tem um **identificador único** e a sincronização é idempotente: se o mesmo lote chegar duas vezes ao servidor, é gravado uma só vez. O registo local só é apagado depois de o servidor confirmar a receção, o que garante que nenhuma leitura se perde.



Como funciona a Iris



Os números (tempo no intervalo, médias, episódios e previsões) são sempre calculados pelo backend e pelo modelo de previsão. A Iris, através da API Google Gemini, apenas os transforma em texto: simples para o paciente e para o cuidador, técnico para o profissional de saúde. Os pedidos não levam nomes nem identificadores, e a ligação ao Gemini fica isolada num adaptador, para se poder mudar de fornecedor sem alterar o resto do sistema.

5. Estudo de viabilidade tecnológica

A tabela justifica cada escolha e indica a alternativa que foi considerada e porque foi posta de lado.

Tecnologia	Vantagens	Alternativa considerada
Kotlin + Jetpack Compose (Android)	Linguagem oficial do Android; acesso direto a Bluetooth LE, Health Connect e serviços em segundo plano; é a tecnologia da UC de Computação Móvel, onde a app Android também é avaliada	Flutter: uma só base de código, mas não cumpre o requisito de Kotlin nativo da UC
Swift + SwiftUI (iOS)	Linguagem oficial da Apple; acesso nativo a Core Bluetooth e HealthKit; boas ferramentas de acessibilidade para utilizadores idosos	Manter Flutter só para o iOS: uma ferramenta extra para uma única plataforma
Go	Rápido e leve; lida bem com muitas leituras em simultâneo; binário único, fácil de pôr em Docker	Node.js: mais conhecido, mas menos eficiente sob carga contínua
Python + FastAPI	Ecossistema de referência para modelos de previsão; exporta modelos leves para o telemóvel	Treinar em Go: bibliotecas de ML muito mais limitadas
PostgreSQL	Relacional e robusto para contas, permissões e histórico; preparado para séries temporais	MongoDB: pior para relações entre pacientes, cuidadores e empresas
SQLite	Base de dados local, sem servidor; ideal para a fila de sincronização	Guardar em ficheiros: sem garantias de integridade
Bluetooth Low Energy	Padrão usado pelos sensores; consumo de bateria muito baixo; funciona sem Internet	Ler só da app do fabricante: dados com atraso
Health Connect / HealthKit	Única via oficial para ler relógios de várias marcas com uma só integração	Integrar marca a marca: inviável
Google Gemini	Plano gratuito para desenvolvimento; boa qualidade em português; troca de fornecedor isolada num adaptador	Modelo próprio no servidor: lento sem placa gráfica
FatSecret Platform API	Base de dados alimentar com porções e hidratos por porção prontos a usar; plano Premier Free gratuito para estudantes	Criar a base de dados de alimentos de raiz: inviável para dois elementos
FCM + APNs	Serviços oficiais e gratuitos de notificações push para os alertas aos cuidadores	SMS: custo por mensagem



Tecnologia	Vantagens	Alternativa considerada
OAuth 2.0 / OIDC	Logins externos sem guardar palavras-passe de terceiros; padrão da indústria	Só login próprio: pior experiência de registo
WebRTC	Videochamada no browser e na app, cifrada, sem software extra	Link para Zoom ou Teams: sai da aplicação
Oracle Cloud + Docker	Servidor gratuito com recursos suficientes; contentores iguais em teste e produção	Azure: créditos de estudante limitados no tempo
Jenkins	Open source, sem custos de licença; corre no próprio servidor	GitHub Actions: válido, mas menos controlo sobre a infraestrutura

6. Exemplo prático: servidor na Oracle Cloud

O servidor da solução já foi criado e configurado. Os passos seguidos foram os seguintes.

- **Criação da máquina virtual:** instância Ampere A1 (ARM) com 4 núcleos, 24 GB de memória e 150 GB de armazenamento.
- **Sistema operativo:** Ubuntu Server LTS.
- **Rede:** rede virtual (VCN) com regras de entrada apenas para as portas 22 (SSH), 80 e 443 (HTTPS).
- **Acesso:** ligação por SSH com par de chaves, sem autenticação por palavra-passe.
- **Serviços:** instalação do Docker, onde vão correr a API, o serviço de previsão, a base de dados e o Jenkins.

[Inserir captura de ecrã da consola da Oracle com a instância criada, ou do terminal com a ligação SSH]

7. Modelo de dados

A base de dados PostgreSQL tem **29 tabelas normalizadas** (terceira forma normal), organizadas em sete módulos. O **paciente** é o centro do modelo: a telemetria, a nutrição, as previsões, os alertas e o serviço clínico ligam-se a ele por chave estrangeira. O script SQL completo foi testado em PostgreSQL e é entregue em anexo (voltik_schema.sql).

Na revisão do modelo foram acrescentadas três tabelas, marcadas como “nova” na figura: **identidades_externas** (logins Microsoft, Google, Apple e Facebook), **consentimentos** (registo RGPD do que cada utilizador aceitou) e **tokens_notificacao** (envio de alertas por notificação push aos cuidadores).



PK chave primária · FK chave estrangeira → tabela de destino · UK valor único · "nova" = tabela acrescentada na revisão

Figura 2 — Modelo de dados completo (29 tabelas)

Privacidade Os dados de saúde só são acessíveis ao próprio paciente e a quem ele autorizou em vinculos_cuidador ou em atribuicoes_paciente_profissional. A empresa nunca acede a valores individuais, e todos os acessos a dados de saúde ficam registados em auditoria_acesso_dados.



8. Custos

Na fase de desenvolvimento o projeto tem custo zero. Os custos seguintes só surgem na publicação nas lojas de aplicações ou com utilizadores reais em escala.

Recurso	Desenvolvimento	Publicação e produção
Servidor Oracle Cloud	Gratuito	Gratuito dentro dos limites do plano; servidor pago de cerca de 5 € por mês como alternativa
API Google Gemini	Plano gratuito	Pago por utilização: cerca de 0,1 cêntimos por relatório
Jira, GitHub, Jenkins, Docker	Gratuito	Gratuito para equipas pequenas
FatSecret Platform API	Premier Free (estudantes)	Premier pago para dados de Portugal e códigos de barras de produtos portugueses
Firebase Cloud Messaging / APNs	Gratuito	Gratuito
Apple Developer Program	Não necessário	99 USD por ano, obrigatório para publicar na App Store
Google Play Console	Não necessário	25 USD, pagamento único
Domínio e certificado	Opcional	Cerca de 10 € por ano; certificado TLS gratuito

9. Riscos e mitigação

Risco	Mitigação
Os sensores comerciais usam Bluetooth cifrado e proprietário	A camada Bluetooth é desenvolvida e testada contra um simulador próprio; a fonte de dados é trocável por um adaptador, preparando a integração oficial futura
Poucos dados reais para treinar o modelo	Datasets públicos de investigação (OhioT1DM, D1NAMO) e dados sintéticos gerados por simulador
O plano gratuito do Gemini pode usar os dados enviados	Só dados simulados e de testadores que assinaram consentimento; nunca são enviados nomes nem identificadores; migração para plano pago antes de dados reais em escala
Alteração das condições do plano gratuito da Oracle	Tudo corre em Docker, com cópias de segurança diárias: a mudança para outro servidor faz-se em poucas horas
Enquadramento como dispositivo médico	Fase académica apresentada como ferramenta de bem-estar e segurança; o caminho para a marcação CE fica documentado
Os planos gratuitos da FatSecret só incluem a base de dados dos EUA	Alimentos genéricos da base norte-americana, complementados por uma tabela própria de pratos portugueses (fonte_origem = portugues_tradicional)
Duas aplicações nativas duplicam o trabalho de interface	Lógica de negócio concentrada na API em Go e contrato OpenAPI partilhado; a app Android avança primeiro e a app iOS segue o mesmo desenho



Risco	Mitigação
Equipa de dois elementos e prazo curto	Âmbito priorizado por MoSCoW; sprints semanais para detetar desvios cedo

10. Conclusão

O Voltik é **tecnologicamente viável**. Todas as tecnologias escolhidas são maduras, gratuitas na fase de desenvolvimento e dominadas ou em aprendizagem pela equipa. O servidor já está operacional, a arquitetura garante o requisito crítico de funcionar sem Internet, e os riscos identificados têm uma mitigação concreta. Os únicos custos previstos surgem na publicação nas lojas e na passagem a um plano pago de IA, ambos compatíveis com uma fase de investimento futura.